数据库系统

数据：数据库中存储的基本对象，描述事物的符号记录.

数据库(DB)：长期储存在计算机、有组织、可共享大量数据集合.

数据库管理系统(DBMS)：

①提供数据定义语言，②DBMS要分类组织、存储和管理各种数据(字典、用户、数据及其存取路径)

③用户使用DML操纵数据.

④控制DB的建立、运用、维护

DBMS运行控制功能：安全性保护、数据完整性检查、并发控制、数据恢复.

数据库系统 DBS.
- DB.
- DBMS（及其开发工具）
- 应用系统、管理员.

数据模型（DM）：对现实世界数据特征的抽象/模拟.

DM分类.
- 概念模型（用于DB设计）
- 逻辑&物理模型

逻辑模型
- 层次模型
- 网状模型
- 关系模型
- 面向对象 & 对象关系 model.

物理模型：数据最低级抽象，描述Data表示方式、存取方法.

DM 组成（三要素）
- 数据结构
- 数据操作
- 完整性约束（条件）

数据管理发展.

手工阶段（银行业务管理）

程序阶段（早期应用程序处理）

文件阶段（文件系统）

数据库阶段

开发步骤

需求分析 → 概念设计 → 逻辑设计 → 物理设计 → 系统实现 → 运行&维护.

DBS的三级模式结构

- 外模式 → 视图、部分表（用户）
- 概念模式 → 全局逻辑结构（联系）
- 内模式 → 物理存储细节
  （文件组织、索引、分区、压缩、块大小）

优点：

① 逻辑独立性：概念模式变化时，外模式可以不变。

② 物理独立性：内模式变化（换哈希索引、重新分块……）不影响概念模式

③ 系统灵活性：多视角支持，更安全、DBA管理简化。

概念模型

① 用于信息世界建模，与具体DBMS
② 现实→信息第一层抽象
③ DB设计工具
④ 设计人员与用户间的交流语言.

实体.
客观存在、可相互区别的事物

属性
对实体特征的描述.

码（关键字）.
唯一标识实体的属性集.

域
具有相同数据类型的值的集合.

#实体型

具有相同属性实体共有特征

#实体集

同一类型实体的集合.

#联系

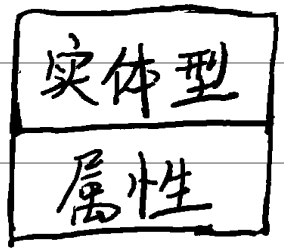
实体间的关联

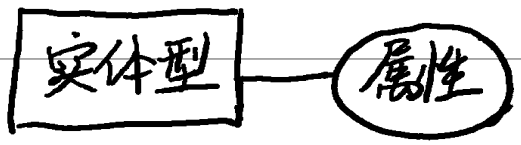
① 1:1联系：实体集A、B一一对应

② 1:多联系：A中每一实体联系B中多个实体.

③多:多联系

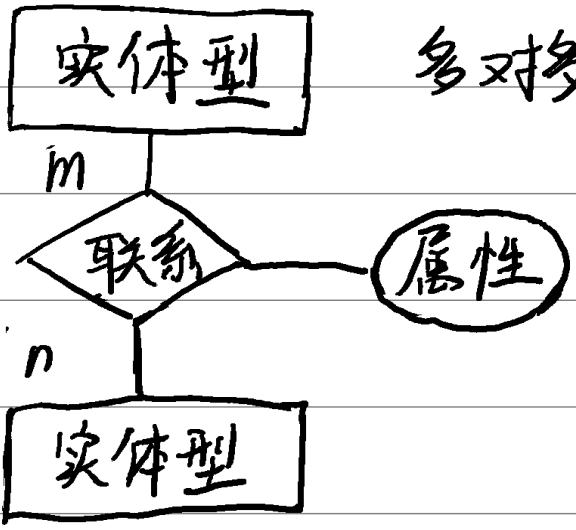
§☆E-R模型



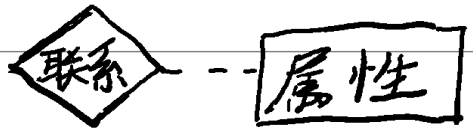


标志属性划下划线

key



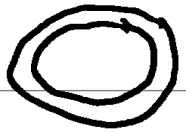
多对多联系



全参与为双线

部分参与为单线

复合属性：由其它属性构成

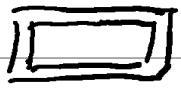
多值属性：可以同类型有多个 

导出属性：可由其它属性推导.

键(属性)：每个实体都独有. 下划线标注

三元联系集 ⇒ 二元：将联系实体化，再分别联系

弱实体集

弱实体集 

弱联系 

弱实体集：①没有键

②存在依赖于主实体集

③键由主实体集键和它的分辨符合并构成.

多值属性 & 复合属性

处理

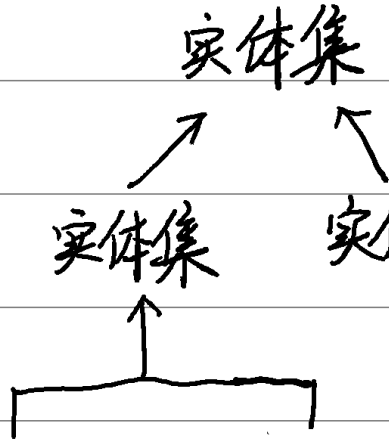
① 将原多值属性用多个单值属性代替

② 将原多值属性用一个实体代替.

特化与概化

父实体集可能有子实体集

特化：自顶向下；概化：自底向上



重叠特化.

不相交特化.

*加上 total 表示全参与，全部概化：
每个父实体集必属于某一子实体集.

另一种画法 ψ: is a. o: 重叠特化
d: 不相交特化.

#聚集
将联系集看作更高级的抽象实体集

层次模型 -> 树

网状模型 -> 图

*关系模型 -> 表格

1. 非层次结构 -> 层次结构

① 采用副本（冗余）

② 虚拟记录（指针开销大）

存储结果：邻接法（先序）.

操作：get unique / next / next within parent.

2. 网状模型

联系的表示（系set）{单属 多属

存储结构：单/双向链接，环状，向首链拉.

操作：find、get、store、modify、erase、(dis/re)connect

§关系模型（关系是笛卡尔积的子集）

① 关系：一个关系对应一张表

② 元组（Tuple）：表中的一行

③ 属性（Attribute）：表中的一列.

多个元组 ⇒ 实例.

1NF

①多值属性 ⇒ 纵向展开.

phone number
①，②

⇒

phone number
①
②

②复合属性 ⇒ 横向展开

住址 ⇒ 省市 区县 街道 门牌号

超码

可以成为惟一标志元组的属性集

候选码

最小超码称为候选码

主码

人为用于区分不同元组的候选码（人为指定的）

一个关系中仅允许有一个主码

外码

关系模式的属性存在另一个关系模式 r，r 称为外码

模式图

- 数据模型：描述数据对象内部结构、相互关联
- 线条：外键—参照关系，描述对象间关联.

• 矩形：关系模式，描述对象特征

主码完整性约束：关系数据库中必须有一个主键，主键的值不能重复，也不能为空.

约束

外码参照完整性：表与表之间的关联必须保持一致；不允许出现“无父类”的子记录。

域完整性：根据实际情况，定义表中属性的取值范围.

参照完整性：不引用不存在的实体.

§ 关系运算

σ（选择）返回满足谓词的行

Π（投影）对所有行输出指定属性，去重

$\bowtie$（自然连接）左/右/全外连接

$\times$（笛卡尔积）相同时，两个都要保留

$\cup$（并集）

ρ（重命名）

e.g：学生关系 S（Sno，Sname，age，sex）

学习关系 SC（Sno，Cno，grade）

课程关系 C（Cno，Cname，teacher）

① 检索学习课程号为 C2 的学生 Sno，grade

$$\Pi_{Sno,grade}(\sigma_{cno=c2}(SC))$$

② 检索 Cno为C2学生的 Sno，Sname

$$\Pi_{Sno,Sname}(\sigma_{SC.Sno=S.Sno}(\sigma_{Cno=C2}(SC)\times S))$$

③ 检索 Cname为 Maths 的 Sname，Sno.

$$\Pi_{Sno.Sname}(\sigma_{SC.Cno=C.Cno \wedge SC.Sname=S.name}(\sigma_{Cname=Maths}(C)\times SC\times S))$$

④ 检索 Cno为 C2或C4的 Sno.

$$\Pi_{Sno}(\sigma_{Cno=C2}(SC)\mid\sigma_{Cno=C4}(SC))$$

§ 外连接运算

① 左外连接

instructor ⟕ teaches

全保留　　只保留匹配的

② 右外连接

instructor ⟖ teaches

只保留匹配的　　全保留.

③ 全外连接

instructor ⟗ teaches

全保留　　全保留

④ 自然连接

只保留两者相匹配的

广义投影（拓展）

$\Pi_{ID, name, dept_name, salary/12}$（as 重命名）

使用表达式　　年薪⇒月薪

聚集函数

avg　min　max　sum　count（元组数）

gfunc（属性）（关系）.

g count（*）（关系）.

E-R图 ⇒ 关系模型

简单实体集 → 新关系模式

属性集 & 主码 = 实体集属性集 & 主码

弱实体集 → 新关系模式

1) 属性集：弱实体集的属性集 +
主实体集的主码

2) 主码：主实体集主码 +
弱实体集分辨符

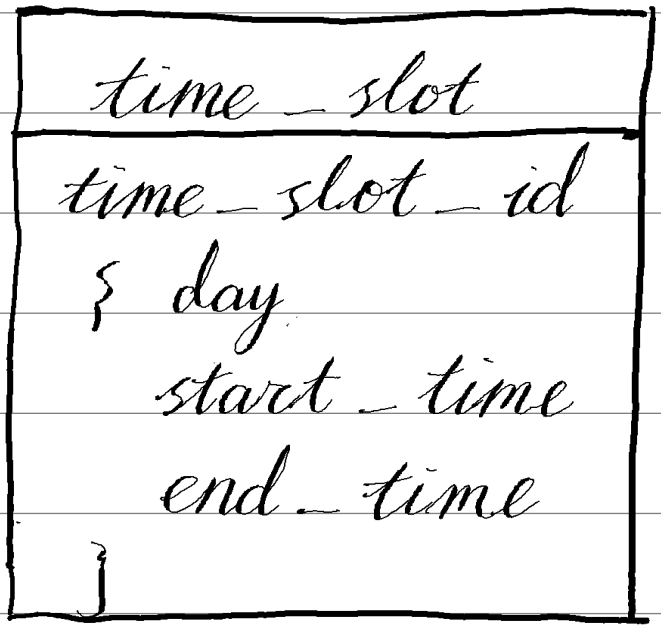
复杂实体集 → 新关系模式

1) 属性集：简单属性 + 复杂属性展
开转换为多个简单属性.

2) 主码：根据惟一确定元组特征

time_slot_id , day
start_hr , start_min
end_hr , end_min

⇒



#联系转换.

1:M 1方主键出现在M方成为外键

1:1 任一方主键成为另一方外键

M:N 联系建为一新关键.其主键由两个父实体集主键复合而成

三元联系 先转关系模式

三端实体类型键 + 联系属性

一元联系转换同二元

基本 SQL

create table -n .
(属性 数据类型,
primary key ( ));

外键
foreign key ( ) reference -n ( )

where 子句.
select name. 投影
from instructor .
where dept_name = ... 选择.

Union 垂直合并

Except 差.

Intersact 交.

group by 数据分组，
常与聚合函数并用.

```
select dept_name , avg(salary) as
avg_salary
from instructor
group by dept_name
having avg(salary) > 60000;
```

按部门求平均薪资

筛选

嵌套

where 嵌套

```
select name, salary
from instructor
where salary > (
        select avg(salary)
        from instructor
        where dept_name = ''
);
```

#删除

```
delete from instructor
where dept_name = '';
```

嵌套

```
delete from instructor
where dept_name in (select dept_name
```

```
from department
where building = '');
```

插入

```
insert into course (course_id, ··· )
values ('CS-437', ··· )
```

null 为空值.

```
insert into instructor
        select ID, ···, 18000
    from student
        where ·····
```

从学生表中选择满足的记录，插入导师表中. (学生变成了老师)

更新数据

```
update instructor
set  salary = salary * 1.03
where  salary > 100000;
```

使用 case 语句

```
update instructor
set salary = case
        when salary <= 100000 then
                salary * 1.05
        else salary * 1.03
        end;
```

5 连接

① 迪卡尔积筛选

```
select ... , ...
from ... , ...
where ... = ...
```

ID 出现 2 次

② 自然连接

```
select ... , ...
from ... natural join ... ;
```

*③ using 筛选

```
select ... , ...
from (... natural join ... )
join course using (...);
```

*④ on 连接

```
select *
from ... join ... on ... = ... ;
```

高级 SQL.

1、视图

create view as ...

可视化

2.1. 完整性约束

属性非空、唯一、范围完整性约束

复杂条件

check (semester in (...))

2.2 外键约束

1) 不写时拒绝删除.

2) 连带删除(修改)

```
create table course (                    (P.73-74)
...
dept_name varchar(20),
foreign key (dept_name) references department
        on delete cascade          1)
        on update cascade,         2)
...);
```

set null / set default

设空 / 默认值

2.3 断言

create assertion <> check <>

当数据更新时，保持谓词为真，否则拒绝.

3.1 表关系授权

grant select/update/… on instructor to U_1, U_2, U_3

revoke 收回 (all)

通过授权，只有授权用户才能实现操作.

- **grant select on** *department* **to** Amit **with grant option**;
- **revoke select on** *department* **from** Amit, Satoshi **cascade**;
- **revoke select on** *department* **from** Amit, Satoshi **restrict**;

① 允许转授

② 级联收回

③ 防止级联收回

3.2 视图上授权

create view geo_instructor as
(...);

grant select on geo_instructor
to geo_staff

将视图上的查看权限授予

3.3 授权图

grant update

4. SQL函数

① create function dept_cnt ( )
returns
begin ...
end

嵌套

```
create function instructors_of (dept_name char(20)
        returns table (  ID varchar(5),
                         name varchar(20),
                         dept_name varchar(20),
                         salary numeric(8,2))
return table
        (select ID, name, dept_name, salary
         from instructor
         where instructor.dept_name = instructors_of.dept_name)
```

```
select *
from table (instructors_of ('Music'))
```

② SQL过程

create procedure dept_cnt_proc()

一段SQL语句程序 → void

无返回值.

③ 外部语言

create procedure ... ( )

language C

external name 'path'.

5. 触发器.

```
create trigger credits_earned after update of takes on (grade)
referencing new row as nrow
referencing old row as orow
for each row
when nrow.grade <> 'F' and nrow.grade is not null
     and (orow.grade = 'F' or orow.grade is null)
begin atomic
     update student
     set tot_cred = tot_cred +
          (select credits
           from course
           where course.course_id = nrow.course_id)
     where student.id = nrow.id;
end;
```

案例10.a 使用触发器来维护 *credits_earned* 值
(student表的属性)

① 当属性值改变时

② 改变前后记录比较

可用于复制或备份数据库

优化 - 范式与函数依赖.

§冗余：数据某些部分能由其它部分推导出来，说明有冗余.
(存在约束，函数依赖)

优化：模式分解.

S	N	L	R	W	H
123-22-3666	Attishoo	48	8	10	40
231-31-5368	Smiley	22	8	10	30
131-24-3650	Smethurst	35	5	7	30
434-26-3751	Guldu	35	5	7	32
612-67-4134	Madayan	35	8	10	40

无损分解

=

S	N	L	R	H
123-22-3666	Attishoo	48	8	40
231-31-5368	Smiley	22	8	30
131-24-3650	Smethurst	35	5	30
434-26-3751	Guldu	35	5	32
612-67-4134	Madayan	35	8	40

⋈

R	W
8	10
5	7

#函数依赖：关系r中，属性集X中相等，属性集Y值也必须相等.

A	B	C
1	1	2
1	1	3
2	1	3
2	1	2

FDs with A as the left side:	**Satisfied by the relation instance?**
A→A	yes
A→B	yes
A→C	No
A→AB	yes
A→AC	No
A→BC	No
A→ABC	No

FD	Satisfied by the relation instance?
C→B	yes
C→AB	No
B→C	No
B→B	Yes
AC →B	Yes [note!]
…	…

函数依赖可以检测冗余是否存在

§ 范式

第一范式：每个属性为原子属性

第二范式：所有非主属性必须依赖于整个候选码而非部分

（不包含在任何候选键中属性）

§ Armstrong

公理：

- 自反：$Y \subseteq X$，则 $X \to Y$
- 增广：$X \to Y$，则 $XZ \to YZ$
- 传递：$X \to Y$，$Y \to Z$，则 $X \to Z$

合并：$X \to Y$，$X \to Z$，则 $X \to YZ$

分解：$X \to YZ$，则 $X \to Y$，$X \to Z$

伪传递：$X \to Y$，$WY \to Z$，则 $WX \to Z$

*2NF分解.

$R(A, B, C, D)$ 候选键 (A, B)

$F = \{A \Rightarrow C\}$ (C部分依赖)

$R_1(A, C)$

$R_2(A, B, D)$ D完全依赖于AB.

① 对 $A \Rightarrow C$ 新建关系模式 $R_1 = AC$

② 从原关系中去除C，保留A，检查 R_2 (原属性集 − C)

③ 递归

不保证依赖保持

保证无损依赖.

e.g. $R = \{A, B, C, G, H, I\}$

p1. $F = \{A \to B,\ A \to C,\ CG \to H,\ CG \to I,\ B \to H\}$

推出 $A \to B,\ B \to H \Rightarrow A \to H$

$A \to C \Rightarrow AG \to CG$

$AG \to CG,\ CG \to I \Rightarrow AG \to I$

§ 函数依赖集的闭包

用 Armstrong 公理推导出的所有函数依赖的集合 (F^+)

属性集闭包 $(AG)^+$

$CG \to H$

p2. $AG \to BG,\ AG \to CG.$ $CG \to I$

$(AG)^+ = \{ABCGHI\}$ AG是键

result := α;
    repeat
        for each 函数依赖 $\beta \rightarrow r$ **in** F **do**
            begin
                if $\beta \subseteq$ *result* **then** *result* := *result* $\cup$ γ;
            end
    until (*result* 不变)

e.g2. $F = \{A \rightarrow B,\ AC \rightarrow D,\ AB \rightarrow C\}$

$A \rightarrow B \Rightarrow A \rightarrow AB$.

$\Rightarrow A \rightarrow AB \rightarrow C$

同理 $A \rightarrow AC \rightarrow D$.

$\therefore A \rightarrow ABCD$, $A^+ = ABCD$

- $R = (A, B, C, G, H, I)$
  $F = \{A \rightarrow B;\ A \rightarrow C;\ CG \rightarrow H;\ CG \rightarrow I;\ B \rightarrow H\}$
  $(AG)+ = ?$
- Answer: ABCGHI
- *AG 是候选键吗?*
  - 这个问题包括两部分:
    1. AG 是一个超键吗?
       - $AG \rightarrow R?$ == Is $(AG)+ \supseteq R$
    2. AG的子集是否是一个超键?
       - $A \rightarrow R?$ == Is $(A)+ \supseteq R$
       - $G \rightarrow R?$ == Is $(G)+ \supseteq R$

属性闭包作用:

① 测试超键.

X为超键 $\Leftrightarrow$ $X^+ \rightarrow R$.

② 检测函数依赖.

$X \rightarrow Y \Leftrightarrow Y \subseteq X^+$

③ 计算F的函数依赖集闭包

§ 计算 F^+

- F={ A → B, B → C}. Compute F+ (包括属性A, B, C).

A→BC

	A	B	C	AB	AC	BC	ABC
A	√	√	√	√	√	√	√
B		√	√			√	
C			√				
AB	√	√	√	√	√	√	√
AC	√	√	√	√	√	√	√
BC		√	√			√	
ABC	√	√	√	√	√	√	√

Attribute closure
A^+=ABC
B^+=BC
C^+=C
AB^+=ABC
AC^+=ABC
BC^+=BC
ABC^+=ABC

- 每一个√ 表示FD (行) → (列) 在 F+中.
- 每一个√ (列) 在(行)+中

由图可知 A 为超键 (候选键)

无损连接分解.

$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$

分解后再连接，和原来一样.

充要条件 $(R_1 \cap R_2) \to R_1$

或 $(R_1 \cap R_2) \to R_2$.

§ BCNF 分解.

模式 R 满足 BCNF 范式

- 对所有非平凡函数依赖 $X \Rightarrow Y$，X 均为超键.

无损分解 → 检查 BCNF

(否) → 继续分解.

BCNF 分解 ① 结果可能不同.

② 函数依赖可能无法保持

符合 BCNF 范式不会产生冗余

3NF (第三范式)

允许存在一定冗余.

既持无损，又保持函数依赖.

所有平凡函数依赖 $X \to A$

① 符合 BCNF 范式. (X为超键) or

② A是键的一部分

分解：使用下最小函数依赖集 (FC)

#正则覆盖 (F_C)

$\alpha \to \beta \Rightarrow (\alpha - A) \to \beta$

则 A 为多余属性

① $\alpha_1 \to \beta_1$, $\alpha_2 \to \beta_2$, $\alpha_1 = \alpha_2$

合并 $\alpha_1 \to \beta_1\beta_2$.

② 删除多余属性

3NF 分解算法

① 求 F_c

② F_c 每个依赖做成一子模式 R_i

③ 候选键作子模式

§ 设计目标

① 优先 BCNF + 双保持

② 其次考虑 3NF 分解产生冗余.

e.g.1

$R(A, B, C, D, E, F)$

$F = \{A \to C,\ C \to A,\ B \to AC,\ D \to AC\}$

1. 候选键 BDEF.

2. 不符合 3NF.

$F_c = \{A \to C,\ C \to A,\ B \to A,\ D \to A\}$

$R_1 = (AC)$ 已合并.

$R_2 = (AB)$

$R_3 = (AD)$

$R_4 = (BDEF)$

e.g 2 $R = (A, B, C, G, H, I)$

$F = \{A \to B,\ A \to C,\ CG \to H,\ CG \to I,\ B \to H\}$.

$F_C = \{A \to BC,\ CG \to HI,\ B \to H\}$

$R_1(ABC)$ $F_1 = \{A \to BC\}$

$R_2(CGHI)$ $F_2 = \{CG \to HI\}$

$R_3(BH)$ $F_3 = \{B \to H\}$

$R_4(AG)$ $F_4 = \phi$

物理设计任务：考虑用文件表示逻辑数据模型（数据库模式）的不同方式。

#定长记录

无论实际存储数据多长，每条记录都占用相同字节数，如果数据长度小于固定长度，会用填充字符补齐.

- 文件中记录的组织方式
  - 堆文件：记录在文件空间中任意放置
  - 顺序文件 ：按一定的顺序在文件中组织记录
  - 散列文件：按照散列函数计算值存放相应记录
  - 多表簇集文件：不同关系表里的记录存放在同一个文件中。

1. 顺序文件组织

① 关系中记录按搜索码次序进行物理存储

② 采用定长记录或变长记录方式

③ 一个记录不能分存在2个物理块

2. 多表聚集文件组织

将多个关系组织在一个文件中.

(需底层操作系统配合)

缺点:查询较麻烦(需要指针

数据字典

关系模式和其它元数据存储称为数据字典/系统目录的结构中。

数据库 $\xrightarrow[1:n]{\text{包含}}$ 表 $\xrightarrow[1:n]{\text{包含}}$ 属性

$\xrightarrow[1:n]{\text{占有}}$ 块 $\xrightarrow[1:1]{\text{对应}}$ 数据库. (RDBMS)

Create database ⇒ 创建容器

Create table ⇒ 在容器里创建结构

Insert 找到目标块，插入，若已满，再申请新块

Delete 标记数据为Delete 更新索引，空间标记可重用.(物理不擦除)

Update 原地修改.

缓冲区：缓解 CPU与磁盘速度的矛盾.

读操作流程：

用户查询 → 缓冲区有数据吗?

├─ 有（命中）→ 直接返回 → 极快

└─ 无（未命中）→ 从磁盘读入缓冲区 → 再返回

写操作流程：

用户修改 → 先写入缓冲区（标记为“脏页”）→ 立即返回成功

→ 后台线程择机将脏页刷回磁盘

内容	说明
数据页	表的实际数据行所在的块
索引页	索引结构的块（B+树节点）
自适应哈希索引	热点数据的哈希映射（如 InnoDB）
插入缓冲	临时存放二级索引的变更
锁信息	行锁、表锁等
日志缓冲区	还未刷盘的 REDO LOG

替换策略（缓冲区满了，读入新页，踢出旧页）

策略	核心思想	适用场景
LRU	淘汰最近最少使用的	通用，有局部性规律
MRU	淘汰最近最常使用的	某些特殊场景（一次扫描）
LFU	淘汰使用频率最低的	访问频率差异大
FIFO 先进先出	淘汰最早进入的	简单实现，但不准
Clock	LRU 的近似实现	性能开销小，广泛使用
随机	随机淘汰	极简单，性能尚可

Clock：指针循环扫描，访问位=1，清零，继续；访问位=0，淘汰该页。

索引与散列

{ 顺序索引
  散列索引



搜索码 指针(→记录文件)

稠密：每个搜索码都有一个索引
稀疏：只为某些搜索码建立索引

稠密索引，次序可不同；
稀疏索引，次序需相同。

多级索引（基于内层索引建立外层索引）
索引文件上再建索引。

辅助索引（非聚集型索引，须稠密）

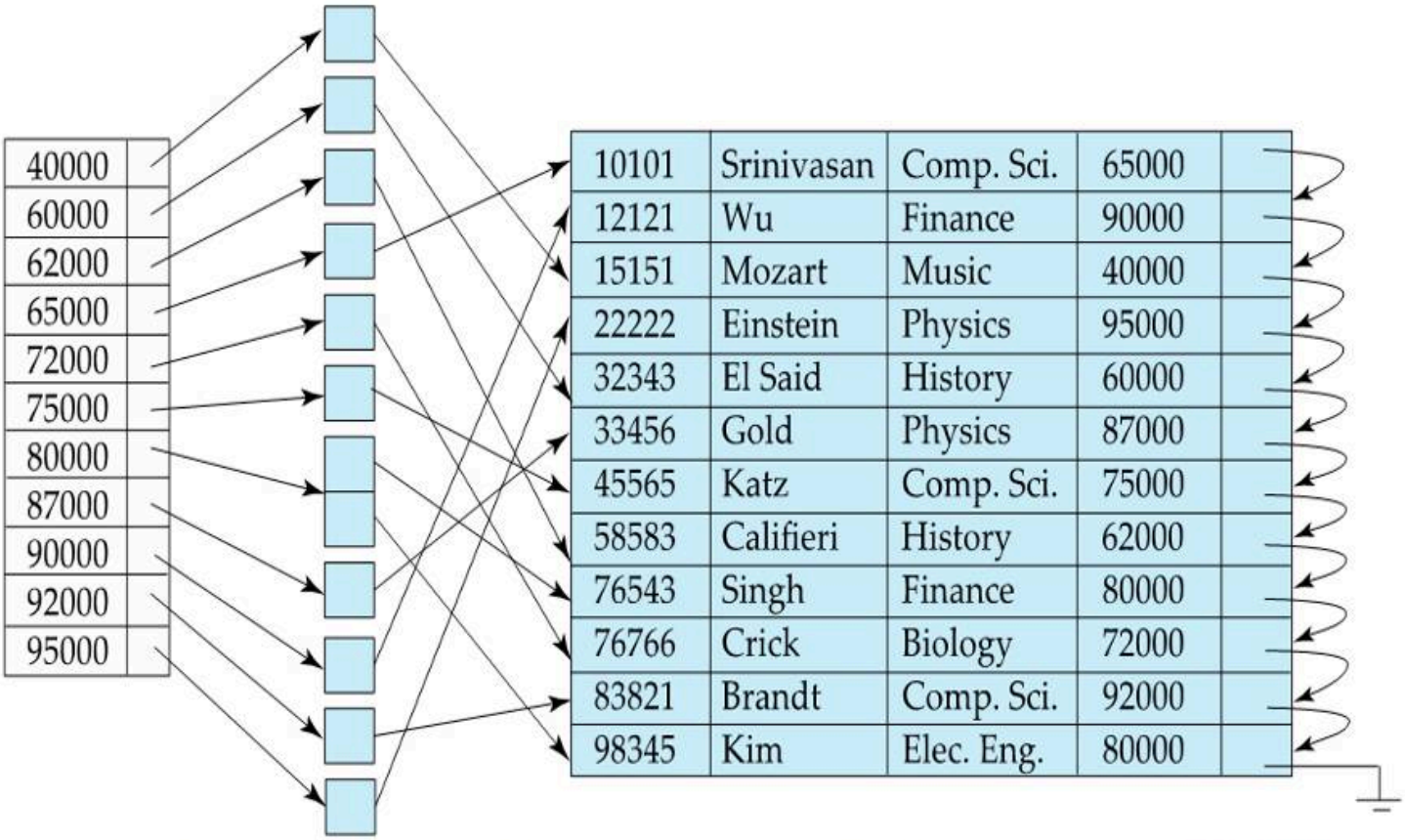
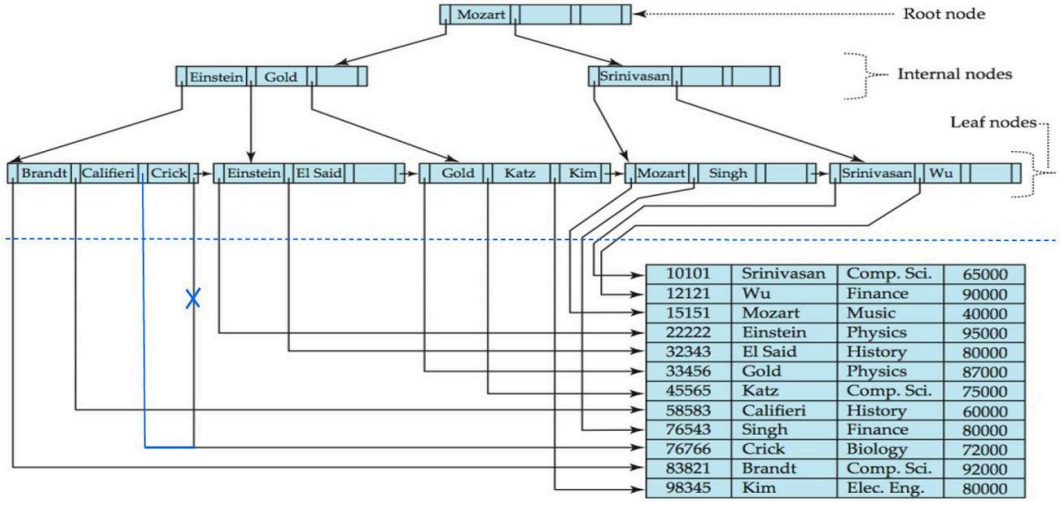


图11-6 instructor的辅助索引（基于非候选码salary-索引码）

* B^+ 树索引

（平衡树，根到任意叶路长相同）



插入更新

① 节点未满，只需直接将搜索码填入相应节点

② 原节点已满需一分为二.

删除更新.

① 节点半满，直接删除.

② 节点太空，调整相应节点(合并)

§ 散列索引

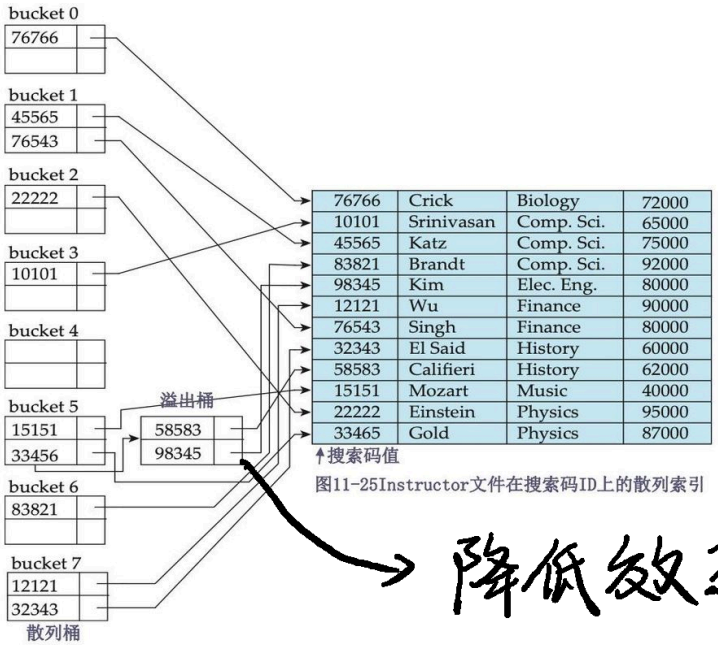


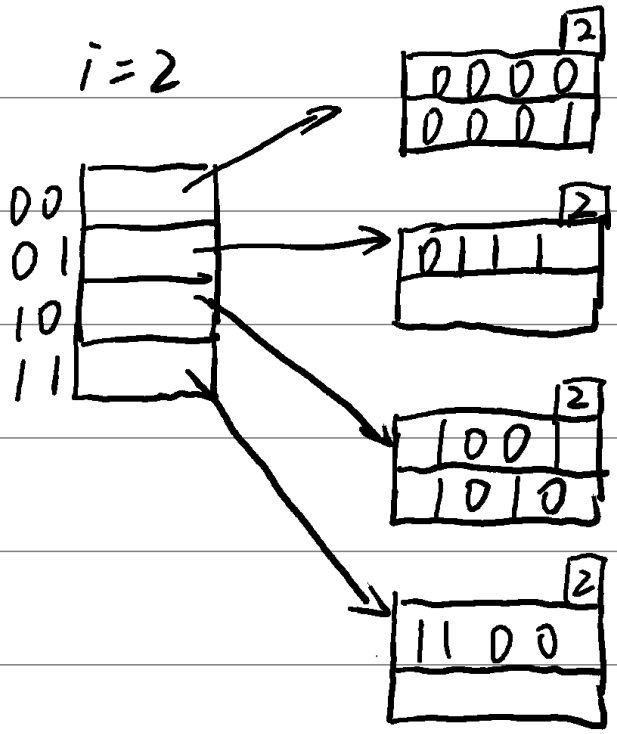
图11-25Instructor文件在搜索码ID上的散列索引

降低效率

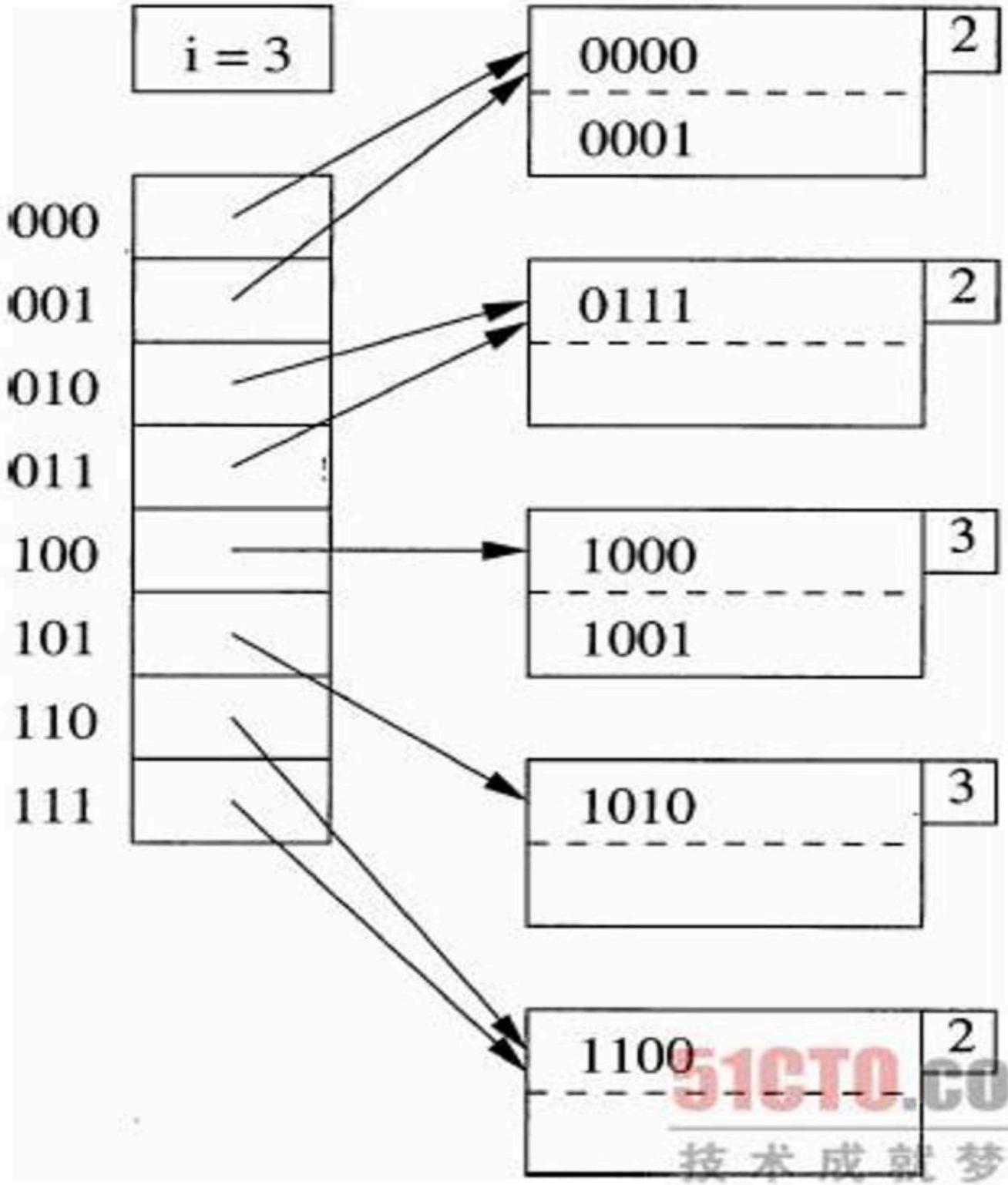
动态散列

桶满先扩展地址再加桶.

散列桶数目不固定，通过散列前缀 i (此≤i) 控制



插入1000

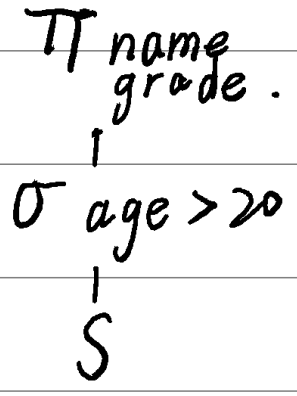


51CTO.com
技术成就梦想

查询处理.

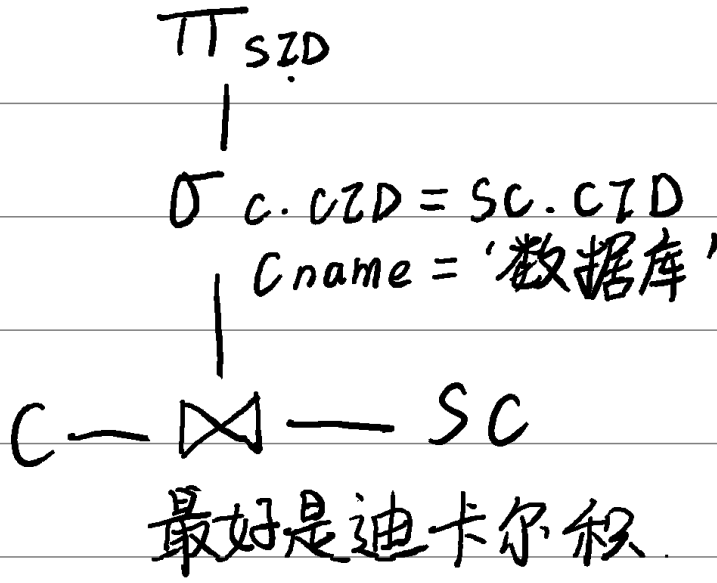
[e.g1] select name, grade
from S
where age > 20;

关系代数式 $\Pi_{\substack{name \\ grade}}(\sigma_{age>20}(S))$



[e.g2] select SID
from C, SC
where C.CID = SC.CID and

Cname = '数据库';



最好是迪卡尔积

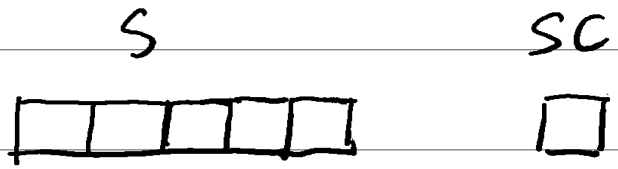
执行开销

① 磁盘存取块数 I/O 代价

② CPU 代价

③ 查询代价

① $Q_1 = \Pi_{sname}(\sigma_{S.sno = SC.sno \atop SC.Cno = '2'}(S) \times SC)$



1000 S. 10000 SC.

每块10个 每块100个

$20 \times 100 + 100 = 2100$ 块.

连接后元组数 10^7 个. 每块10个.

10^6 块.

② $Q_2 = \Pi_{sname}[\sigma_{SC.Cno = '2'}(S \bowtie SC))$

读取 2100 块

自然连接后 10^4 条 $\Rightarrow$ 10^3 块.

③ $Q_3 = \Pi_{sname}(S \bowtie \sigma_{SC.Cno = '2'}(SC))$

先读100块 $\Rightarrow$ 50个记录(选择)

读S 100块

代数优化

① 先选择再连接

② 投影下移

③ N:关系先连接

物理优化（索引/全表查询）

查询优化（查询重写）

逻辑代数上的优化

① 常量表达式化简

② 子查询优化（不相关可独立求解）

③ 选择下移，推理等价

④ 外连接消除.

⑤ distinct消除.（减少不必要计算）

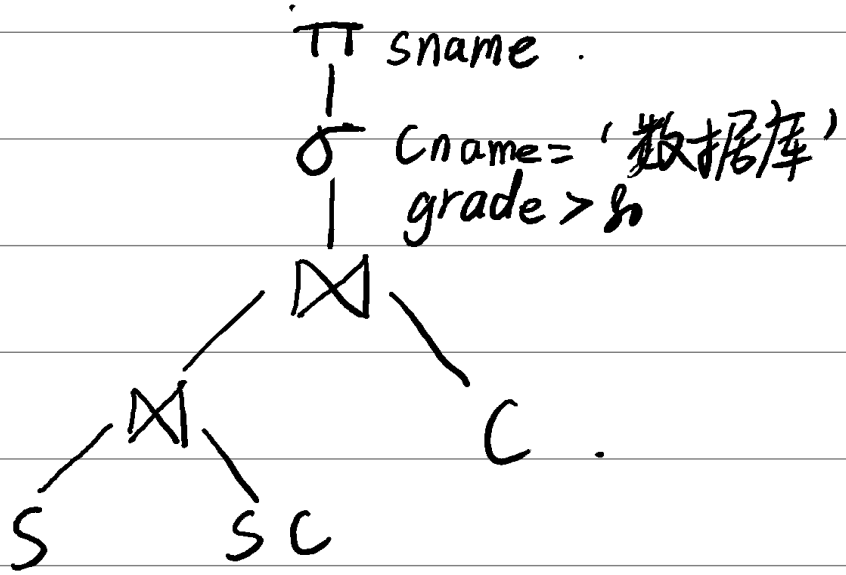
⑥ in 谓词展开

⑦ 视图展开

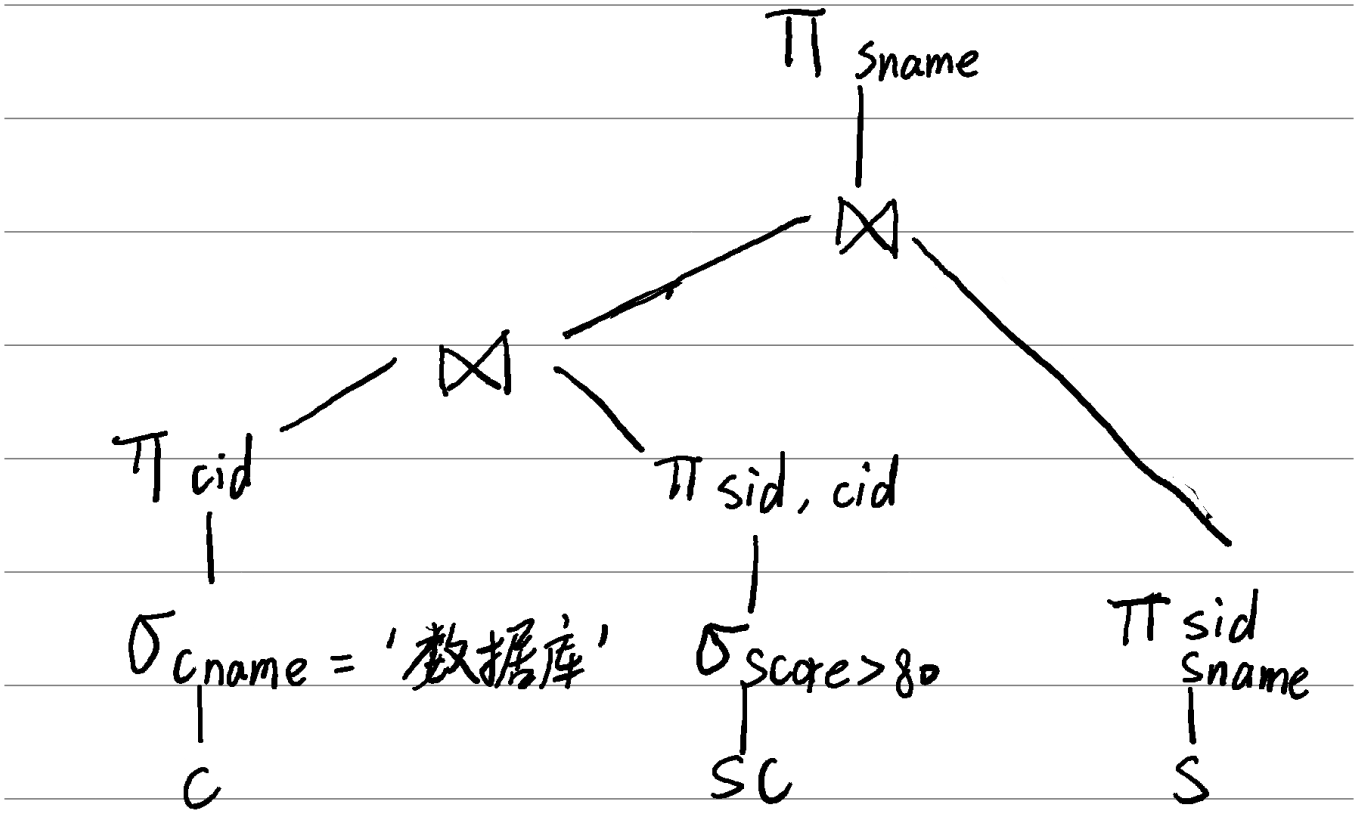
e.g.

```
select  S.Sname
from  S, SC, C
where  S.SID = SC.SID  and
       C.Cname = '数据库'  and
       SC.grade > 80;
```

$$\Pi_{Sname}(\sigma_{Cname='数据库' \wedge grade>80}(S \bowtie SC \bowtie C))$$



查询优化.

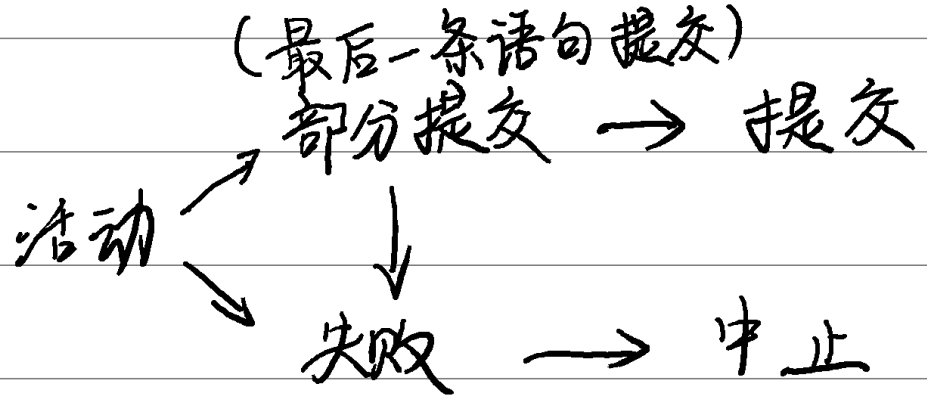


事务管理

构成单一逻辑工作单元的操作集合。

① 一致性　② 原子性

③ 持久性　④ 隔离性



调度：一组指令包括指令在系统中执行的特定时间顺序

串行～：按序依次执行

并行～：同时执行

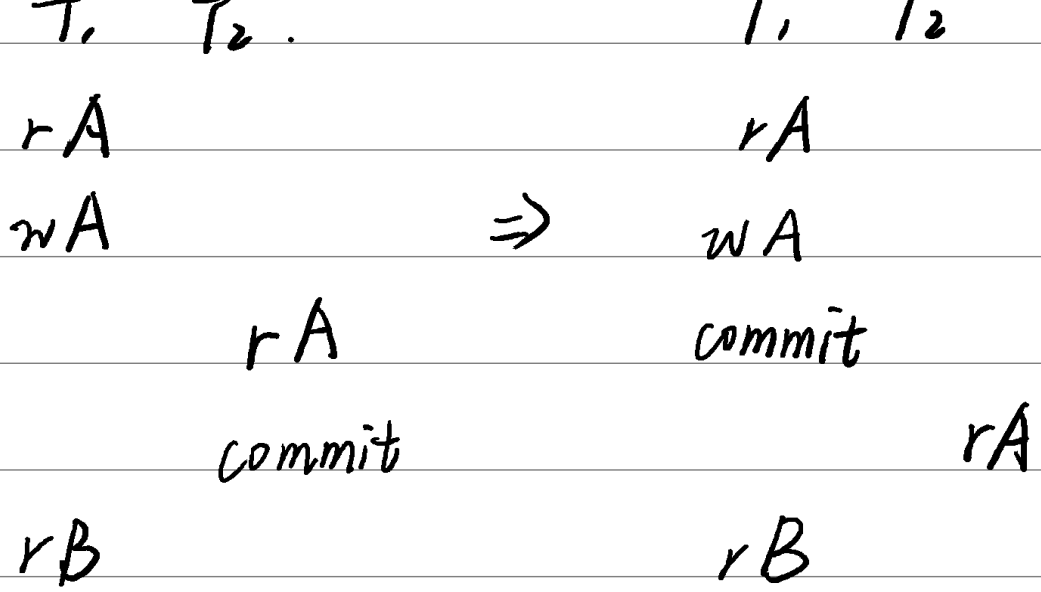
调度 S 通过一系列非冲突指令交换转化成 S'，则 S 和 S' 冲突等价

调度优先图

有环：非冲突可串行化

read → write

可恢复调度：T_i 先提交，T_j 后读取



画图步骤

1. **节点**：为调度中的每个事务画一个节点。
2. **边**：对于任意两个**冲突的操作**（属于不同事务，访问同一数据项，且至少一个是写操作），如果操作 o_i 在 o_j 之前执行，则画一条有向边 $T_i \to T_j$。
   - **冲突类型**：
     - T_i 写 Q，T_j 读 Q（写-读冲突）
     - T_i 读 Q，T_j 写 Q（读-写冲突）
     - T_i 写 Q，T_j 写 Q（写-写冲突）
3. **检查环**：若图中存在**有向环**，则调度**不是冲突可串行化**；若无环，则**是冲突可串行化**。

无级联调度：事物只读取已提交的数据，不允许脏读.（相对宽松）

SQL 标准规定的隔离性级别如下。

- **可串行化**(serializable)：通常保证可串行化调度。然而，正如我们将要解释的，一些数据库系统对该隔离性级别的实现在某些情况下允许非可串行化执行。
- **可重复读**(repeatable read)：只允许读取已提交数据，而且在一个事务两次读取一个数据项期间，其他事务不得更新该数据。但该事务不要求与其他事务可串行化。例如：当一个事务在查找满足某些条件的数据时，它可能找到一个已提交事务插入的一些数据，但可能找不到该事务插入的其他数据。
- **已提交读**(read committed)：只允许读取已提交数据，但不要求可重复读。比如，在事务两次读取一个数据项期间，另一个事务更新了该数据并提交。
- **未提交读**(read uncommitted)：允许读取未提交数据。这是 SQL 允许的最低一致性级别。

幻读 前后不一致(插入)
不可重复读 脏读 事务回滚.

只允许读提交 → 脏.
严格两阶段协议 → 不可重复.

锁

lock-s 共享锁.

只读数据，不修改.

lock-x 排他锁.

可读&写

#2阶段封锁协议.

阶段一：事物只获得锁

阶段二：事物只释放锁

仅有封锁未必能保证调度的可串行化。

#死锁

两个事物互相等待.

T_3	T_4
lock-x (B)	
read (B)	
$B := B - 50$	
write (B)	
	lock-s (A)
	read (A)
	lock-s (B)
lock-x (A)	

活锁：一个事务持续等待某个资源，无法进展.

2PL ⇒ 可串行化（充分不必要）

两阶段封锁协议

严格～：事物持X锁，提交后须释放

强～：提交前不得释放任何锁.

锁转化

lock-s (a1)

update (a1) S锁升级为X锁

e.g. read(a), read(b), read(c)
c = a + b, write(c)

lock-S(a).
read(a)
lock-s(b)
read(b)
lock-X(c)
c = a+b.
write(c)
unlock ...

基于图的协议（偏序）

树形协议 ⇒ 冲突可串行化

⇒ 无死锁

不保证无级联卷回

除首次，加锁需父结点有锁，解锁后不能再上锁。

死锁（资源分配图有环等待）

① 选择牺牲

② 回滚 { 彻底
部分.

预防：① 执行前给所有数据项加锁

② 强加次序

③ 使用抢占和事务回滚

{ 新等老
老等新.

多粒度锁

IS：表明将在下层节点加S锁.

IX：.....X锁.

SIX：S+IX.

	IS	IX	S	SIX	X
IS	true	true	true	true	false
IX	true	true	false	false	false
S	true	false	true	false	false
SIX	true	false	false	false	false
X	false	false	false	false	false

图15-16 相容性矩阵-多粒度

优势

① 提高并发度：允许多事务访问同一表的不同行，并发度高

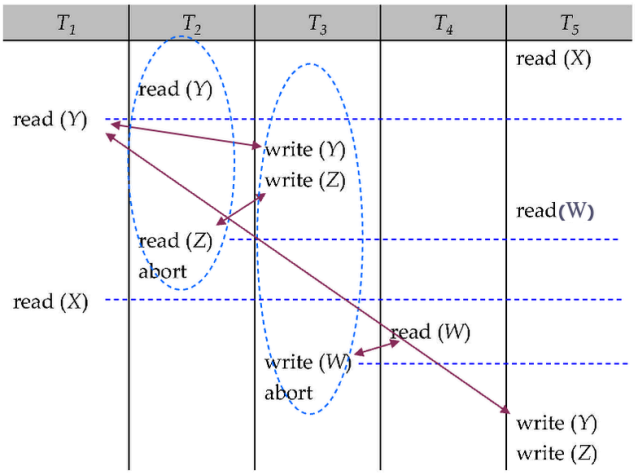
② 减少锁开销：表锁只需一个锁封锁整个表.

时间戳排序协议(timestamp-ordering protocol)保证任何有冲突的 **read** 或 **write** 操作按时间戳顺序执行。该协议运作方式如下：

注：以下均为Ti

1. 假设事务 T_i 发出 **read**(Q)。
   a. 若 TS(T_i) < W-timestamp(Q)，则 T_i 需要读入的 Q 值已被覆盖。因此，read 操作被拒绝，T_i 回滚。
   b. 若 TS(T_i) ≥ W-timestamp(Q)，则执行 read 操作，R－timestamp(Q)被设置为 R-timestamp(Q)与 TS(T_i)两者的最大值。
2. 假设事务 T_i 发出 **write**(Q)。
   a. 若 TS(T_i) < R-timestamp(Q)，则 T_i 产生的 Q 值是先前所需要的值，且系统已假定该值不会再产生。因此，**write** 操作被拒绝，T_i 回滚。
   b. 若 TS(T_i) < W-timestamp(Q)，则 T_i 试图写入的 Q 值已过时。因此，**write** 操作被拒绝，T_i 回滚。
   c. 其他情况，系统执行 **write** 操作，将 W-timestamp(Q)设置为 TS(T_i)。

如果事务 T_i 由于发出 **read** 或 **write** 操作而被并发控制机制回滚，则系统赋予它新的时间戳并重新启动。



TS(T_i)=i, i=1 ,…, 5

执行过程分析？

R-timestamp(X)=5正常读
R-timestamp(Y)=2正常读
R-timestamp(Y)=2正常读(不更新)
W-timestamp(Y)=3正常写(需检查)
W-timestamp(Z)=3正常写
R-timestamp(W)=5正常读
异常读，撤消事务T2(需检查) 回滚.

R-timestamp(X)=5正常读(不更新)
R-timestamp(W)=5正常读(不更新)
异常读,撤消事务T3(需检查)

W-timestamp(Y)=5正常写(需检查)
W-timestamp(Z)=5正常写

未发生：时间戳为 0.

冲突可串行化，无死锁，调度优先图无环.

5 快照隔离

按照**先提交者获胜**(first committer wins)方法，当事务 T 进入部分提交状态，以下操作作为一个原子操作执行：

- 检查是否有与 T 并发执行的事务，对于 T 打算写入的某些数据，该事务已经将更新写入数据库。
- 如果发现这样的事务，则 T 中止。
- 如果没有发现这样的事务，则 T 提交，并且将更新写入数据库。

- 每个事务开始时被分配一个**时间戳**（或事务ID）。
- 数据库为每个数据项维护**多个历史版本**，并标记每个版本的“创建事务”和“删除事务”的时间戳。
- 读操作只读取**开始时间戳之前已提交**的版本，因此**不会看到其他未提交事务的修改**（无脏读），也**不会看到本事务开始之后其他已提交事务的修改**（可重复读）。
- 写操作（更新/删除）会创建新版本，但**提交前**会检查是否存在与其他同时期事务的**写-写冲突**（即是否有另一个事务在相同时间戳区间内修改了同一数据项）。若冲突，则后者提交失败（回滚）。

不保证可串行化

避免脏读、不可重复读.

核心规则

当两个并发事务试图修改**同一数据项**时：

- 两者都基于各自启动时的快照进行修改（都能成功写入本地版本）
- 在**提交时**，系统检查是否存在写-写冲突
- **先提交的事务成功提交**
- **后提交的事务被中止（回滚）**，并收到“序列化失败”错误

恢复 & 日志

Log	Write	Output
$<T_0$ start$>$		
$<T_0$, A, 1000, 950$>$		
$<T_0$, B, 2000, 2050$>$		
	$A = 950$ $B = 2050$	
$<T_0$ commit$>$		
$<T_1$ start$>$		
$<T_1$, C, 700, 600$>$		
	$C = 600$	
		B_B, B_C
$<T_1$ commit$>$		
		B_A

前项-修改前的值

后项-修改后的值

（系统）执行时间轴

- **Note:** B_X denotes block containing X.

常见3种情况

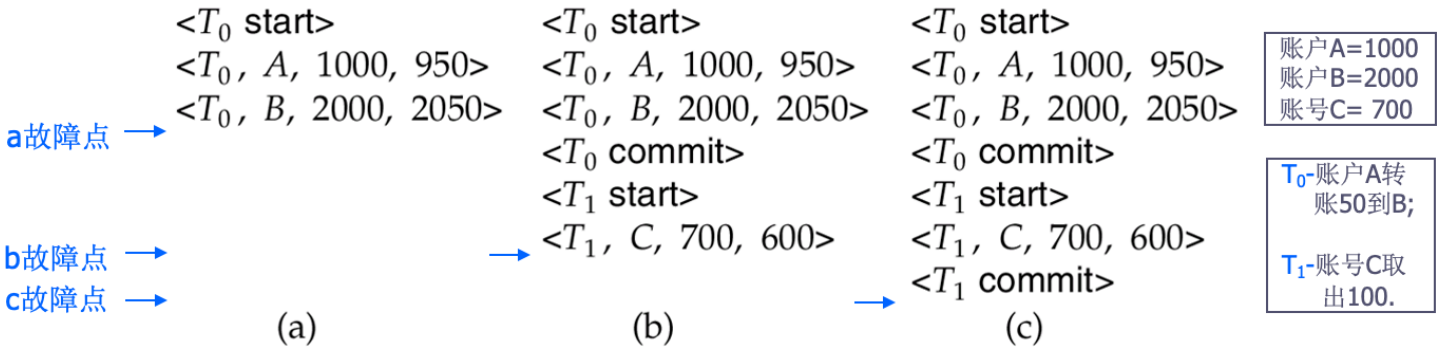


图 16-4 在三个不同时间显示的同一个日志

a) 均未提交 ⇒ undo

<T_0, B, 2000> <T_0, A, 1000>

<T_0, abort>

b) 仅 T_0 提交, redo T_0,

undo T_1. (redo 不写日志)

<T_1, C, 700> <T_1, abort>

c) 均已提交, redo T_0, T_1

E.G. 含 abort

T_0 redo

T_1 undo



```
<T0 start>
<T0, A, 800, 700>
<T0, B, 1000, 1100>
<T1 start>
<T1, C, 800, 600>
<T0, B, 1000>
<T1, D, 600, 800>
<T0, A, 800>
<T0, abort>
```

→ 执行完了, redo.

后次后写：<T_1, D, 600>

<T_1, C, 800>

<T_1, abort>

§ checkpoint 检查点

<checkpoint, T>

T为当前活跃事务

checkpoint前提交的事务，不进行 redo / undo